

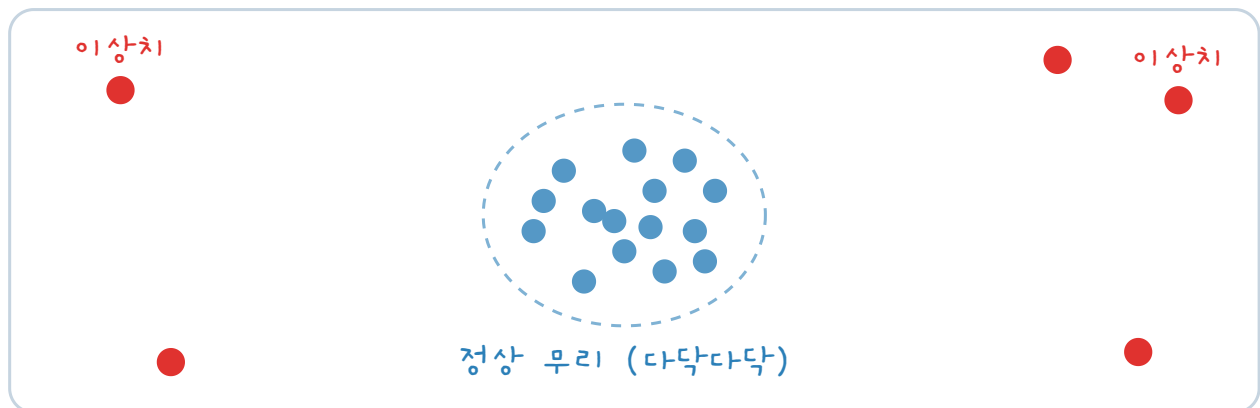
이상치탐지

정상에서 유별나게 벗어난 소수를 정답 없이 찾아내는 일.
사기 거래·불량품·장비 고장을 잡아냅니다.

Q1 이상치가 뭔데?

이상치(anomaly)는 말 그대로 **무리에서 유별나게 튀는 하나**입니다. 반 평균 키가 165인데 혼자 200인 학생, 매달 5만 원 쓰다가 갑자기 500만 원이 결제된 카드 같은 것이죠.

핵심은 **정답표(라벨)가 없다**는 점입니다. "이건 이상, 저건 정상"이라고 미리 알려준 사람이 없어요. 데이터만 보고 "남들과 확 다른 것"을 스스로 찾아내야 합니다. 그래서 **비지도학습**입니다.



가운데 파란 무리 = 정상, 멀찍이 떨어진 빨간 점 = 이상치.

이상치 탐지 = "이 데이터, 남들과 확 다른 게 있나?"

무엇이 이상인지 아무도 안 알려줍니다. 유별남을 스스로 찾죠.

Q2 어디에 쓰나

상 황	찾 아 내 는 이 상 치
카드 결제 내역	평소 패턴과 딴판인 사기 거래
공장 센서·설비 데이터	불량품, 고장 조짐이 보이는 장비
서버 접속 로그	비정상 접근, 침입 시도
수집한 원본 데이터	입력 실수로 튀어버린 값(정제 대상)

공통점은 **"이상**은 드물고, **미리 라벨이 없다**"는 것입니다. 사기 거래가 정상 거래만큼 많지도 않고, 매번 사람이 "이건 사기"라고 표시해 줄 수도 없죠. 그래서 **정답 없이 유별난 소수를 자동으로 찾는** 도구가 필요합니다.

Q3 IsolationForest — 스무고개로 몇 번 만에 혼자 남나

가장 많이 쓰는 방법입니다. 이름 그대로 **"고립(isolation) 시키는 숲(forest)"** 이에요. 원리는 스무고개 게임과 똑 같습니다.

데이터를 **아무 기준으로나 무작위로 뚝뚝 잘라** 나갑니다. 그리고 각 점에게 물어요 — **"너, 몇 번 만에 혼자 남았니?"**



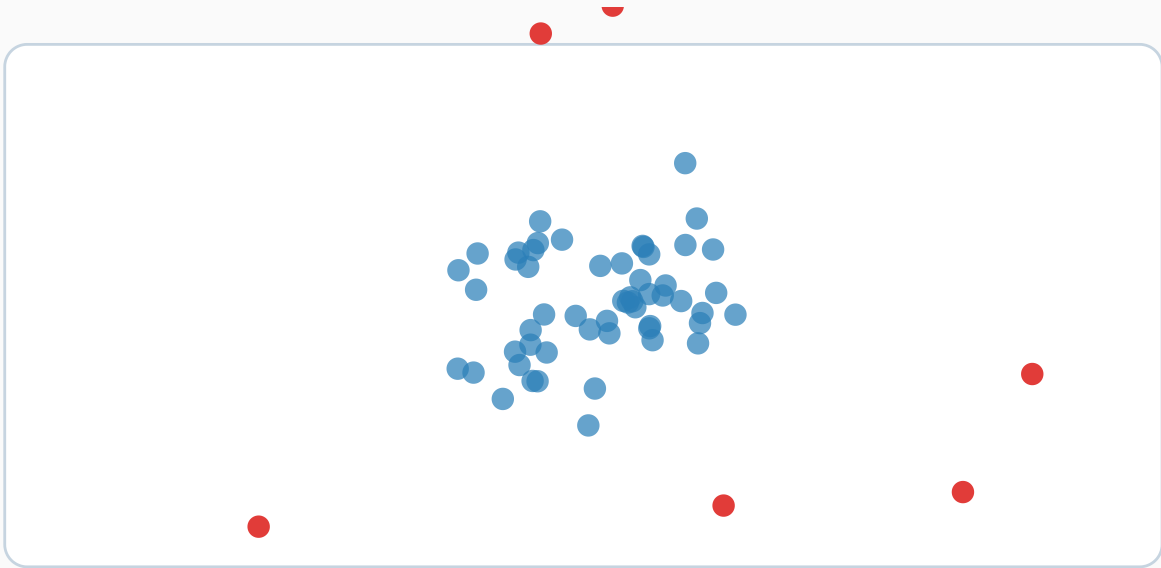
붐비는 곳의 점은 여러 번 잘라야 콧 집히고, 멀리 혼자 있는 점은 **몇 번 만에 고립됩니다**. **빨리 고립될수록 이상치**.

쉽게 말하면 — 붐비는 교실 한가운데 있는 학생은 "창가야?/앞쪽이야?/키 커?"를 여러 번 물어야 콧 집힙니다. 하지만 혼자 복도 끝에 서 있는 학생은 **두세 번 만에** 바로 특정되죠. 그 '빨리 특정됨'이 바로 이상치의 신호예요.

이상치 탐지 실험실

가운데 정상 무리와 멀리 떨어진 이상치 몇 개를 흠어냈습니다.

이상치 비율(contamination)을 조절하면, 그만큼 가장 유별난 점들을 빨강으로 찍어냅니다.



이상치 비율 contamination

10%

이상치로 찍힌 개수

6개

데이터 다시 뿌리기

적당해요. 멀리 떨어진 점들이 이상치로 잘 잡힙니다.

비율을 너무 크게 올려보세요. 멀리 있던 진짜 이상치뿐 아니라 가운데 정상 무리까지 빨강게 물듭니다. 이게 contamination을 크게 잡으면 안 되는 이유예요.

Q5 contamination — 몇 %로 잡을까

contamination은 "전체 중 이상치가 대략 몇 %야?" 라고 알고리즘에게 미리 귀뜸해 주는 값입니다. 이 값만큼을 이상치로 잘라내요.

모르면 작게 잡으세요. 실제 이상치 비율을 알면 그 값을, 모르면 `'auto'` 나 **0.01 ~ 0.1(1~10%)** 정도로 작게 시작합니다.

너무 크게(예: 0.3) 잡으면 **멀쩡한 정상 데이터까지 이상치로 오인합니다** — 위 실험실에서 본 그대로예요.

거리(z-score)로 잡는 방법과 뭐가 다를까?

가장 단순한 방법은 **평균에서 ± 3 표준편차(3σ)를 벗어나면 이상치로 보는 거예요.** 한 개 특성이면 편하지만, 특성이 많아지고 무리가 여러 덩어리로 흩어져 있으면 잘 안 맞습니다.

IsolationForest는 평균·거리를 직접 재지 않고 **"얼마나 빨리 고립되나"**로 판단해서, **특성이 많고 모양이 복잡한 데이터에도** 잘 작동합니다.

Q6 두 가지만 기억 — 스케일링과 -1/1

IsolationForest도 특성 간 **크기 차이에 영향을 받습니다.** 연봉(수백만)과 방문 횟수(한 자리)가 섞이면 큰 숫자가 판단을 지배해요. 그래서 **스케일링**을 먼저 합니다.

```
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest

X_s = StandardScaler().fit_transform(X)    # 같은 잣대로 맞추기

iso = IsolationForest(contamination=0.05, random_state=0)
pred = iso.fit_predict(X_s)                # 학습+판정 한 번에

# 결과는 -1(이상) / 1(정상) 두 종류뿐!
X_normal = X_s[pred == 1]                  # 정상만 남기기 (데이터 정제)
```

결과 읽는 법이 핵심입니다.

IsolationForest는 각 점을 **-1(이상치)** 또는 **1(정상)**로 돌려줍니다. 헷갈리기 쉬워요 — **-1이 이상치**입니다.

`(pred == -1).sum()` 으로 **이상치 개수**를 세고, `pred == 1` 로 **정상만 골라** 데이터를 깨끗하게 만들 수 있어요.

Q7 한 장 정리

	분류 (지도)	이상치 탐지 (비지도)
정답(y)	있음	없음
찾는 것	정해진 칸으로 나누기	남들과 다른 소수 골라내기
결과	클래스 이름	-1(이상) / 1(정상)
핵심 설정	—	contamination(이상치 비율, 작게)
주의	—	스케일링 필수 · 비율 크게 잡지 말기

이것만 기억하면 됩니다

- ① 이상치 탐지는 **정답 없이 유별난 소수**를 찾는 비지도학습이다
- ② IsolationForest는 **빨리 고립될수록 이상치**로 본다
- ③ **contamination**은 이상치 비율 — **모르면 작게(1~10%)**
- ④ 결과는 **-1(이상) / 1(정상)**, 스케일링을 먼저 하자

Q9 Level별 코드 작성하기

옆에 주피터 노트북을 열고, 복사 버튼으로 코드를 붙여넣어 따라 해보세요. 쉬운 것부터 실전(캐글 공공데이터)까지 3단계입니다.

코드 해석 ▼을 누르면 한 줄씩 쉽게 풀어 줍니다.

LEVEL 1 단순 — 정상 무리 + 이상치를 직접 만들어 탐지

```
import numpy as np
from sklearn.ensemble import IsolationForest

rng = np.random.default_rng(0)
normal = rng.normal(0, 1, size=(100, 2)) # 정상 100개 (가운데 몰림)
outliers = rng.uniform(-6, 6, size=(8, 2)) # 이상치 8개 (넓게 흩뿌림)
X = np.vstack([normal, outliers]) # 합쳐서 108개

iso = IsolationForest(contamination=0.1, random_state=0)
pred = iso.fit_predict(X) # -1=이상, 1=정상
print('이상치로 탐지된 개수:', (pred == -1).sum())
```

```
import numpy as np / IsolationForest
```

숫자 배열을 다루는 numpy와 이상치 탐지 도구를 불러옵니다.

```
rng = np.random.default_rng(0)
```

난수 생성기를 만듭니다. 0을 넣어 매번 같은 데이터가 나오게 고정해요(재현 가능).

```
normal = rng.normal(0, 1, ...) / outliers = rng.uniform(-6, 6, ...)
```

정상 100개는 0 근처에 몰리게, 이상치 8개는 -6~6으로 넓게 뿌립니다. 실전에선 어느 게 이상치인지 모르지만, 여기선 이해를 위해 일부러 심어둡니다.

```
X = np.vstack([normal, outliers])
```

두 덩어리를 위아래로 쌓아 108개짜리 한 데이터로 합칩니다.

```
iso = IsolationForest(contamination=0.1, ...)
```

"이상치가 대략 10%쯤 있을 거야"라고 귀똥합니다. random_state=0으로 결과도 고정.

```
pred = iso.fit_predict(X)
```

배우면서(fit) 동시에 각 점을 판정(predict)합니다. 결과는 -1(이상) 또는 1(정상).

```
(pred == -1).sum()
```

-1(이상치)로 찍힌 게 몇 개인지 셉니다. 심어둔 8개 부근을 대체로 잡아냅니다.

LEVEL 2 복잡한 데이터 — 특성 13개 와인, 스케일링 + 이상 점수

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest

X = load_wine().data # 와인 성분 13개 (단위 제각각)
X_s = StandardScaler().fit_transform(X) # 같은 잣대로 (크기 영향 줄이기)

iso = IsolationForest(contamination=0.05, random_state=0)
pred = iso.fit_predict(X_s) # -1=이상, 1=정상
score = iso.decision_function(X_s) # 점수 낮을수록 더 이상함

print('이상치 개수:', (pred == -1).sum(), '/', len(X_s))
print('가장 이상한 5개 인덱스:', np.argsort(score)[:5])
```

```
import ... (4줄)
```

배열(numpy), 와인 데이터, 스케일러, IsolationForest를 준비합니다.

```
X = load_wine().data
```

와인 178병의 성분 **13가지**를 가져옵니다. 특성이 많아 눈으로는 이상치를 못 찾아요 — 알고리즘이 필요한 상황.

```
X_s = StandardScaler().fit_transform(X)
```

알코올(12쯤)과 마그네슘(100대)처럼 크기가 판판이라 **평균 0·표준편차 1**로 맞추는 편이 판단을 독차지하지 않도록.

```
iso = IsolationForest(contamination=0.05, ...)
```

이상치를 **5%**쯤으로 가정합니다. 모를 땐 이렇게 작게 시작해요.

```
score = iso.decision_function(X_s)
```

-1/1 판정 말고 **이상함의 정도(점수)**도 받을 수 있습니다. **점수가 낮을수록 더 이상**해요.

```
np.argsort(score)[:5]
```

점수가 낮은 순으로 정렬해 **가장 이상한 5개**의 위치(인덱스)를 뽑습니다. "무엇이 제일 유별난가"를 꼭 집을 수 있어요.

LEVEL 3 공공데이터(UCI) — 도매 고객 중 유별난 소비 고객 찾기

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest

# 공개 URL로 바로 불러오기 (UCI 'Wholesale customers')
url = 'https://archive.ics.uci.edu/ml/machine-learning-
databases/00292/Wholesale%20customers%20data.csv'
df = pd.read_csv(url) # 도매 고객 440명 (다운로드 불필요)
feat = ['Fresh', 'Milk', 'Grocery', 'Frozen', 'Detergents_Paper', 'Delicassen'] # 품목별 연간 지출
X_s = StandardScaler().fit_transform(df[feat]) # 지출 단위가 커서 스케일링

iso = IsolationForest(contamination=0.05, random_state=0)
df['anomaly'] = iso.fit_predict(X_s) # -1=이상, 1=정상

print('이상치 개수:', (df['anomaly'] == -1).sum(), '/', len(df))
print(df[df['anomaly'] == -1][feat].head()) # 유별나게 많이/적게 쓴 고객
```

```
import pandas as pd ...
```

표(CSV)를 다루는 pandas와 스케일러, IsolationForest를 가져옵니다. 실제 데이터는 표로 오니 pandas가 필요해요.

```
url = '...Wholesale customers data.csv' → pd.read_csv(url)
```

도매 고객 440명의 품목별 연간 지출 데이터를 **공개 URL에서 바로** 읽습니다. 주소의 `%20`은 파일 이름 속 '빈칸'을 뜻해요.

```
feat = ['Fresh', 'Milk', ...]
```

신선식품·우유·식료품·냉동·세제·델리 **6개 지출 열만** 골라 씁니다.

```
X_s = StandardScaler().fit_transform(df[feat])
```

품목마다 금액 크기가 달라 같은 잣대로 맞춥니다.

```
df['anomaly'] = iso.fit_predict(X_s)
```

각 고객을 -1(이상)/1(정상)으로 판정해 표에 **'anomaly' 열**로 새로 붙입니다.

```
df[df['anomaly'] == -1][feat].head()
```

이상치로 찍힌 고객만 골라 지출 내역을 봅니다. 특정 품목을 **유별나게 많이(또는 적게)** 쓴 고객들이 잡혀요 — 도매 대형 거래처 같은.